Street Report Triage — Build Guide, Sector by Sector

Oct 17, 2026 · @Siddhanth Hariharan

1. Conversational AI (BlueBubbles + API)

What this sector does: lets a resident have a real back-and-forth conversation (by text message) with an AI that asks for whatever's missing until it has a complete report.

Quick concepts, explained simply:

- **API** = one program asking another to do something, like ordering off a menu instead of cooking it yourself.
- **Webhook** = instead of constantly asking "did anything happen yet?", you give a service your address and it automatically messages you the moment something does — like a doorbell instead of watching the window.
- **BlueBubbles** = free software on a Mac that turns iMessage into something a program can send/receive from, via an API and webhooks.

The real-world setup (what you already did):

1. Installed BlueBubbles Server on a Mac that stays running
2. Set a server password and local network address (e.g. `http://192.168.0.26:1234`)
3. Added a phone number/email to the Apple ID so residents have something to message
4. Skipped Firebase and Private API setup — neither is needed for plain text + photos

The code, piece by piece (`conversation_ai.py`):

- `ConversationSession` — a plain object holding: the phone number, every message so far, and a status (`open`, `complete`, or `incomplete`)
- `IntakeAgent.handle_message()` — called once per incoming message. Steps, in order:
  1. Add the new message to the session
  2. Check the cap (5 exchanges) — if hit, force `incomplete` without calling the AI at all
  3. Otherwise, send the **whole conversation so far** to Claude (not just the latest message — this is what makes it a real conversation)
  4. Claude replies with structured JSON: `status`, `reply_text`, and a `summary` once complete

5. If Claude's reply isn't valid JSON, fall back to "ask again" instead of crashing

The bridge to BlueBubbles (`bluebubbles_bridge.py`):

- A small Flask server with one job: listen at `/webhook` for new messages
- When a message arrives: find or create that sender's `ConversationSession`, call `handle_message()`, send the reply back out through BlueBubbles' own API
- `active_sessions` is just a Python dictionary in memory — fine for a demo, resets if the script restarts

Build and test in this order:

1. Run `conversation_ai.py` alone first — confirms your Anthropic API key works, no BlueBubbles involved yet
2. Run `bluebubbles_bridge.py`, register the webhook (`http://localhost:5005/webhook`) in BlueBubbles, text yourself, and check the raw payload prints correctly
3. Confirm the field names in the code match what you actually see printed — BlueBubbles' exact format can vary slightly
4. Test a full real conversation: vague opener → AI asks for more → you reply with a photo → confirms completion
5. **Fallback if BlueBubbles eats too much time:** a Streamlit chat box calling the exact same `IntakeAgent` — identical AI logic, just swapping the "front door" for a webpage instead of real texts

2. Classification & Analysis

What this sector does: once a conversation is complete, reads everything (text + any photo) and turns it into a decision: what kind of issue, how severe, and where.

The two pieces:

`KeywordBaselineClassifier` — the deliberately dumb version, built first:

- Just a dictionary of categories to keyword lists (e.g. `"pothole": ["pothole", "hole in the road"]`)
- Checks if any keyword appears in the text, returns the category or `"unknown"`
- Cannot read photos at all, and fails on any phrasing it doesn't recognise — this is the point, since you need something simple to compare the AI against

`ReportAnalyzer` — the real AI step:

- Sends the full conversation transcript, plus a photo if one exists, to Claude
- Photo handling: read the file → base64-encode it → add it as a second content block alongside the text in the same message
- Asks for structured JSON back: `category`, `severity`, `confidence`, and a one-line `explanation`
- If confidence comes back low, the system should never guess — flag for manual review instead

Why the comparison matters: run the same 10–15 test reports through both. The baseline will miss anything photo-only or phrased unusually; the AI should catch those. That side-by-side difference is your actual evidence that "AI at the core" is real, not just a nice-to-have.

Build and test in this order:

1. Write and test the baseline alone — feed it 4–5 sample texts, confirm it classifies the obvious ones and fails the tricky ones
2. Write `ReportAnalyzer` on text-only input first, test the same samples
3. Add photo support, test on a photo-only report the baseline couldn't touch
4. Add the confidence field and the "flag, don't guess" rule — test it with a deliberately vague, ambiguous report
5. Run the full batch comparison (baseline vs AI, same inputs) and keep the results — this becomes evidence in your pitch, not just a dev test

3. Databases

What this sector does: stores every finished report in one place, so nothing gets lost between the chat, the classifier, and the council's website.

The tech choice, explained simply: SQLite — a real SQL database that's built directly into Python (`import sqlite3`), with no separate server, no install, no PHP. It's just one file on disk.

The schema (what's stored for each report): a reference number, phone number, the full transcript, category, severity, confidence, location (plus latitude/longitude), whether it's a duplicate of another report, status, which team it's routed to, and when it was created.

The class (`ReportDatabase`), method by method:

- `save_report()` — creates a new row, generates a short reference number like `REF-A1B2C3` (not sequential, so nobody can guess other residents' reference numbers), returns it
- `get_report(report_id)` — looks up one report, or returns nothing if it doesn't exist
- `list_reports()` — returns every report, newest first — this is what the officer dashboard reads from
- `update_status(report_id, new_status)` — changes a report's status (e.g. to `"resolved"`)
- `find_nearby_same_category()` — the duplicate check: pulls every report of the same category, does simple distance maths in Python, returns the closest one within 50 metres if there is one

Why no name is stored: every one of these functions only ever needs the phone number and the reference number — never a name. That's a deliberate minimal-data-collection choice, not something missing.

Build and test in this order:

1. Run `database.py` directly — it has a built-in demo that saves a report, reads it back, checks for a duplicate, marks it resolved, then lists everything
2. Confirm a `reports.db` file actually appears in your folder — that's the whole database, visible as one file
3. Manually save 2–3 reports with slightly different locations, confirm the duplicate check correctly finds (and correctly ignores) the right ones
4. Only once this works standalone, connect it to Sector 1 (saving a real finished conversation) and Sector 2 (saving the real classification result)

4. Notifications, Tracking & Routing

What this sector does: closes the loop back to the resident, and gives the council a real website to act on reports.

`CouncilRouter` — the simplest piece: a lookup table mapping category to team (pothole → Highways, fly-tipping → Environmental Health, streetlight/graffiti → Street Scene, flooding → Highways, anything else → Customer Services).

`NotificationService` — three separate triggers, not one:

1. **"Received"** — fires the moment intake finishes (Sector 1), before classification even completes, so the resident isn't left wondering if it went through

2. **"Completed"** — fires when an officer clicks resolve on the dashboard
3. **`get_status(report_id)`** — callable anytime, pulls the live status straight from the database using the resident's reference number For the demo, each of these just prints or returns a message instead of sending a real text — the logic is identical either way.

`CouncilDashboard` — the actual council website, built as a second Streamlit page:

- Lists every open report, pulled live from `list_reports()`
- Sortable by severity, so the most urgent issues surface first
- Filterable by location or by which team it's routed to
- A "resolve" button per report, calling `update_status()` and automatically triggering the "completed" notification

The safety/emergency branch — required by the brief, not optional: if a report comes back high-severity and genuinely dangerous (a bad flood, a hazardous pothole), the resident's confirmation includes real official guidance — the Environment Agency's flood warnings, or advice to call 999 or the council's emergency line — rather than just quietly joining the normal queue.

Build and test in this order:

1. Build `CouncilRouter` alone — test every category, including the "other" fallback
2. Build `send_received()`, wire it to fire right after Sector 1 completes a conversation
3. Build `get_status()`, test it against a report you saved manually in Sector 3
4. Build the dashboard page — confirm sorting and filtering actually work against 4–5 fake reports with different severities
5. Wire up "resolve" → `update_status()` → `send_completed()`, test the full loop end to end
6. Add the safety-guidance check last, test it with one deliberately dangerous fake report